

Lab 12:

Character Embeddings & Dimensionality Reduction

Today's Agenda

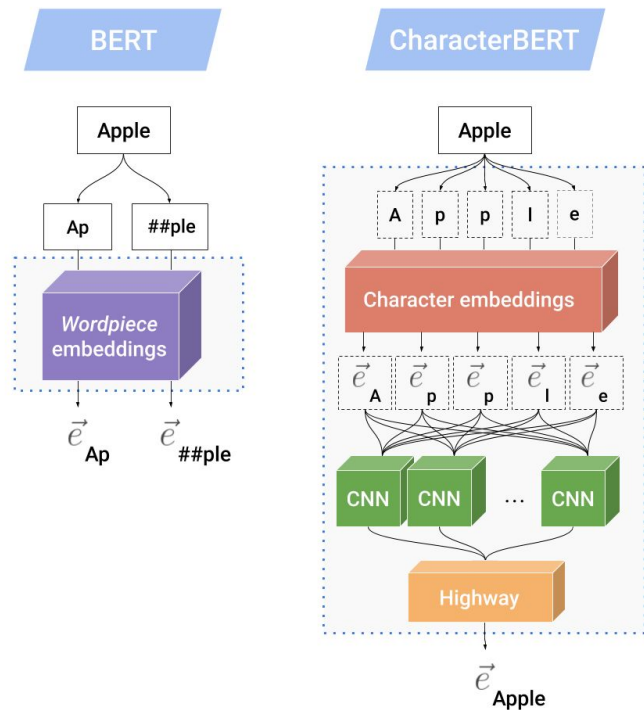
1. Review

- Character Embeddings
- Dimensionality Reduction
 - PCA
 - Autoencoders

2. Jupyter Notebook Examples

Character Embeddings

- Each character in a word represented as a individual dense vector
- Words = sequences of character vectors
- Sequence of vectors can be processed with RNNs/LSTMs, CNNs, Transformers
- Reduces vocabulary size
- Great at learning subword patterns, helpful for rare or misspelled words

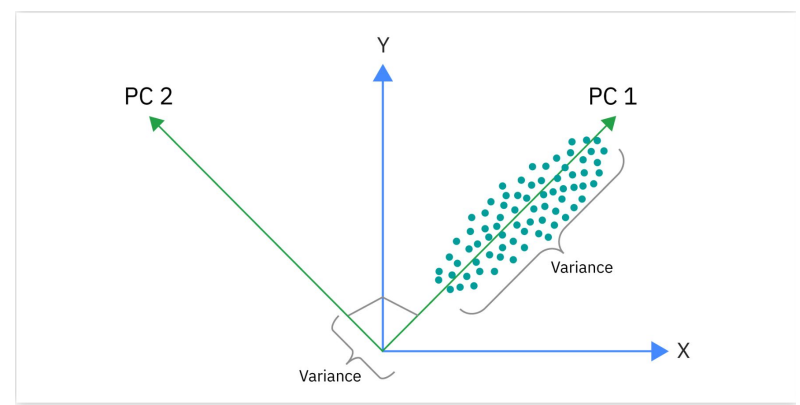


Source:

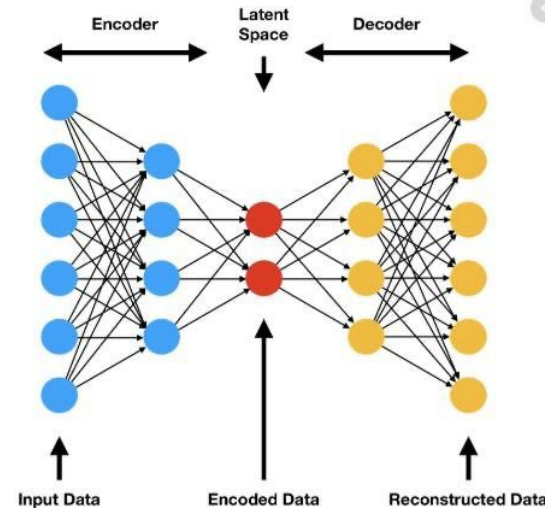
<https://towardsdatascience.com/characterbert-reconciling-elmo-and-bert-for-word-level-open-vocabulary-representations-from-94037fe68b21/>

Dimensionality Reduction

- Reduce number of features, while retaining important details
- Two procedures:
 - Feature selection: Selects the most important features
 - Feature projection: Creates new features by transforming the original ones into a lower-dimensional space



Source: <https://www.ibm.com/think/topics/principal-component-analysis>



Source:

<https://medium.datadriveninvestor.com/dimensionality-reduction-using-an-autoencoder-in-python-bf540bb3f085>

Comparison

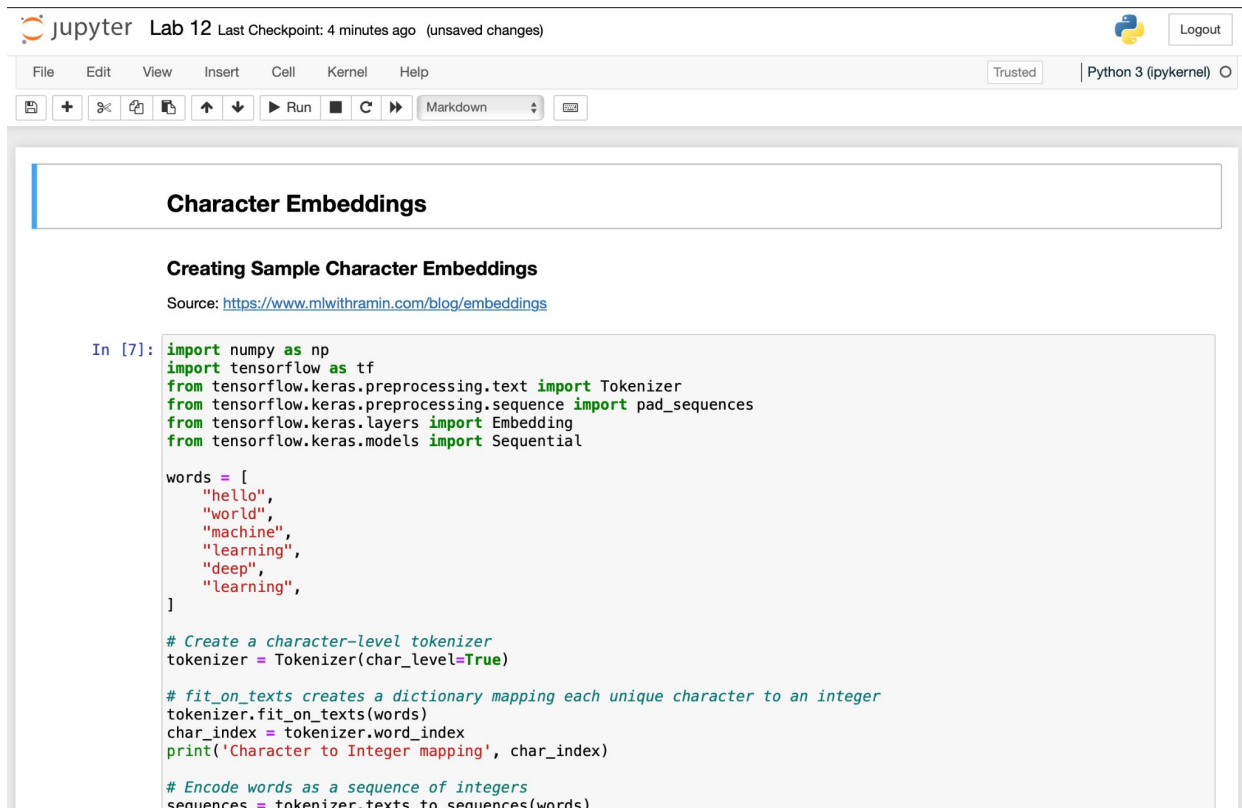
Principal Component Analysis (PCA)

- Project data into a lower-dimensional space using **principal components** (directions of max variance)
- Use **eigenvectors** (principal components) and **eigenvalues** (variance amount) from covariance matrix
- Goal: Maximize variance using orthogonal directions
- Better for linear relationships

Autoencoders

- Type of NN with:
 - **Encoder** to compress input data (image) to lower dimension
 - **Decoder** to reconstruct the original input from reduced encoding
- Goal: Minimize MSE between the reconstructed image and the original image
- Better for nonlinear relationships

Example Notebook



The screenshot shows a Jupyter Notebook interface. At the top, the Jupyter logo is followed by "Lab 12" and "Last Checkpoint: 4 minutes ago (unsaved changes)". On the right, there is a Python logo and a "Logout" button. Below this is a menu bar with "File", "Edit", "View", "Insert", "Cell", "Kernel", and "Help". To the right of the menu bar are buttons for "Trusted" and "Python 3 (ipykernel)". Below the menu bar is a toolbar with icons for saving, adding, undo, redo, and other notebook functions. The main content area has a title "Character Embeddings" in a blue-bordered box. Below the title is a section "Creating Sample Character Embeddings" with a source link: <https://www.mlwithramin.com/blog/embeddings>. The code cell contains the following Python code:

```
In [7]: import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.layers import Embedding
from tensorflow.keras.models import Sequential

words = [
    "hello",
    "world",
    "machine",
    "learning",
    "deep",
    "learning",
]

# Create a character-level tokenizer
tokenizer = Tokenizer(char_level=True)

# fit_on_texts creates a dictionary mapping each unique character to an integer
tokenizer.fit_on_texts(words)
char_index = tokenizer.word_index
print('Character to Integer mapping', char_index)

# Encode words as a sequence of integers
sequences = tokenizer.texts_to_sequences(words)
```